



Cross-platform Framework Comparison

Flutter & React Native

Simon Stender & Hampus Åkesson

18/05/2020

Faculty of Computing
Blekinge Institute of Technology
SE-371 79 Karlskrona, Sweden

This thesis is submitted to the Faculty of Computing at Blekinge Institute of Technology in partial fulfillment of the requirements for the bachelor's degree in software engineering. The thesis is equivalent to 10 weeks of full-time studies.

Contact Information:

Author(s):

Name: Hampus Åkesson

E-mail: hampus.akesson@live.se

Name: Simon Stender

E-mail: simonstender@outlook.com

External advisor:

Name: Young Fogelström

Softhouse employee

University advisor:

Andreas Arnesson

Faculty of Computer Science

Faculty of Computing
Blekinge Institute of Technology
SE-371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

The development of apps in a cross-platform framework is something that has been appearing more over the latest years. But the knowledge of knowing which of the two popular frameworks, React Native, and Flutter are most efficient when it comes to resource management and general comparisons are less known. This is what this thesis investigates.

To find out the comparisons between React Native and Flutter we created two similar apps and document the process of creating an app with the selected frameworks. To get data on the differences when developing an app with these frameworks, we made a survey to get more experienced developers' input. We then did performance tests of the apps to be able to compare the results of the respective framework. The applications we built had several similar functionalities that we used to measure the performance. We also touched on the subject of comparison between a cross-platform framework and a native framework. To do this we performed a literature review on related work to conclude the approaches.

From our result, we could conclude that the performance of the Flutter app had a slight advantage over the React Native app. But the difference was not that remarkable, and the overall development was fairly similar. There were some differences to the approaches of development when it came to less experienced developers compared to more experienced developers which we learn from our survey. More experienced developers tended to use external debugging tools, while less experienced used built-in tools such as console commands.

Finally, we want to conclude that both Flutter and React Native has their pros and cons. Both frameworks have a big community which is growing everyday, but we believe that Flutter might overtake the popularity from React Native due to its slight performance superiority. From our literature review we can conclude that both approaches has their advantages and it depends a lot on the concept of the app. When developing a more complex app, the native approach is superior. When developing smaller apps with shorter life periods, cross-platform will faster get the app on the market.

Keywords: React Native, Flutter, Cross-platform

Contents

1	Introduction	7
1.1	Background	8
1.2	Purpose	9
1.3	Scope	9
1.4	Related Work	10
2	Research Questions	12
2.1	Motivation	13
3	Research Method	14
3.1	Literature Review	14
3.2	Application Experiment	15
3.4	Mobile Application Concept & Code Variation	16
3.4.1	Application Idea	16
3.4.2	Main Application Functionality	16
3.4.2.1	Framework Packages	17
3.4.2.1.1	React-Native	17
3.4.2.1.2	Flutter	18
3.4.3	Debugging	18
3.4.4	Code Complexity	19
3.4.5	Versions	19
3.5	Survey	20

4	Literature Review	21
	4.1 Blueprint App	21
	4.2 Concept	21
	4.3 Design	22
	4.4 Debugging & Testing	22
	4.5 Release	22
	4.6 Future updates in Flutter & React Native	23
5	Results	24
	5.1 Application Experiment	24
	5.1.1 Environment Installation	24
	5.1.1.1 Flutter	24
	5.1.1.2 React Native	24
	5.2 Packages	25
	5.2.1 Flutter	25
	5.2.2 React Native	25
	5.2.3 Package Installation	26
	5.2.3.1 Flutter	26
	5.2.3.2 React Native	26
	5.3 Contents	26
	5.3.1 Flutter	26
	5.3.2 React Native	28
	5.4 Functionality	30
	5.4.1 Flutter	30
	5.4.2 React Native	31
	5.5 Performance Tests	32
	5.5.1 CPU Tests	32
	5.5.1.1 Flutter	32
	5.5.1.2 React Native	35
	5.6 Application Size	38
	5.7 Code Complexity	39
	5.7.1 React-Native Periodic Background Running	39
	5.7.2 React-Native Push-Notification	39
	5.7.3 React-Native Fetch API Communication	40
	5.7.4 Flutter Periodic Background Running	41
	5.7.5 Flutter API Communication	41
	5.8 Survey Results	42

6	Experiment Analysis & Reflection	43
6.1	Development	43
	RQ1 conclusion	43
6.1.1	Code Complexity	43
6.1.2	Debugging	43
6.1.3	Supported External Functionalities	44
6.1.5	Version Handling	44
6.1.5.1	Flutter	44
6.1.5.2	React Native	45
6.1.6	Summary	45
6.2	Cross-platform vs Platform-specific	46
	RQ3 conclusion	46
6.2.1	Native pros:	46
6.2.2	Native cons:	46
6.2.3	Cross-platform pros:	46
6.2.4	Cross-platform cons:	46
6.3	Experiment Analysis	47
	RQ2 conclusion	47
6.3.1	CPU Usage	47
6.3.2	Execution Time	47
7	Conclusion	49
8	Validity Threats	50
	Slight incorrect performance results	50
	Possible database interference	50
9	Future Work	51
	Functionality variety	51
	iOS	51
	Device variety	51
10	References	52
11	Annexes	54

1. Introduction

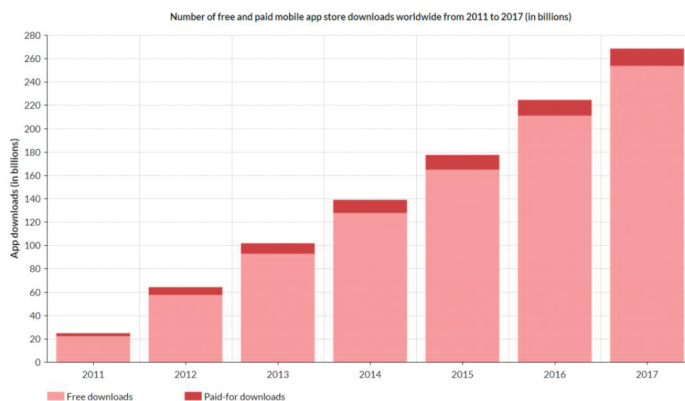
This thesis analyzes two cross-platform development frameworks for mobile applications, Flutter and React Native, and additionally compares the cross-platform approach towards the regular native approach. Therefore, the main idea is to test and analyze the disadvantages and advantages of the respective cross-platform environments where the quality attributes learnability and performance will be the main evaluated aspects.

From these aspects the main advantages to come forward with are how easy it is to learn the chosen frameworks as a beginner, compatibility, and performance differences in two cross-platform frameworks. To test the learnability and performance of the development technologies, we are going to develop two similar apps using the different technologies and document our progress.

Cross-platform in mobile applications is a development approach for software applications that enable a single app codebase to be compatible across multiple devices operating systems such as iOS and Android. Although in this thesis, we will only be doing a comparison on the Android version. The opposite of cross-platform which is native development where the respective applications are built toward a specific devices' operating system.

Mobile application development is something that has only been increasing in popularity during the last couple of years. Due to this popularity, there is a very big market for mobile apps. In 2019 the total revenue for mobile applications all across the world was 461.7 billion dollars.[5]

The Apple App store has 2.2 million apps available for download while the Google Play Store has 2.8 million apps. According to statistics, "49% of people open an app 11+ times each day." [6] Mobile apps can't succeed without downloads, and every year since mobile applications became a thing there have been an increase in downloads, both free apps and paid apps.



Mobile App Download Statistics 2011-2017

Experts predict that there will be a 25% increase in global app downloads between 2019 and 2022, so the growth will not be as significant as it was between 2017-2018, but the market is still growing.

1.1 Background

We chose to compare React Native and Flutter because we came in contact with a software company, which currently has a development team working with React Native, and they want to know more about Flutter because of its recent increase in popularity.

We met an employee at the software company to discuss what research ideas he had regarding mobile development and what could give value to the company. Therefore, this thesis is based on a research idea that the software company is interested in to dive deeper into because of their common use of React Native and curiosity to involve themselves in better solutions and selection of frameworks.

The selected research questions are therefore customized to cover as much as possible of the information and area of interest Young has provided to us.

Both Flutter and React Native are growing in popularity¹ and both of the frameworks are used by many companies around the world today.

But for new companies to know which framework to work with, or for companies who are starting new projects then this thesis will be very useful to make that choice.

React Native was initially released in March 2015[1] compared to Flutter which was initially released in May 2017[2]. This means that React Native has been on the market a while longer and more companies have started working with this framework making a lot of popular apps[3] such as:

- Instagram
- Facebook Ads
- Tesla

Flutter, however, has not been out for as long so there are not as many popular apps made as of yet. But there are still a few popular apps[4] used by millions of people created by Flutter. Such as:

- Google Ads
- Alibaba

¹ “Google Trends” - <https://trends.google.com/trends/explore?date=today%205-y&q=React%20Native,Flutter>, Opened 15 May.. 2020.

1.2 Purpose

The aim is to create guidelines for companies that are interested in mobile development and want to know more about the different approaches to it. With the experiments, we will perform on Flutter and React Native we want to give the reader a proper run-down on which framework suits their needs the best. Due to the fact that Flutter has not been released for as long as React Native has been, some companies can be skeptical about changing the framework if they are already working with React Native. This thesis will hopefully open up the doors for companies to use Flutter if the results we generate are positive.

Another immediate value to the reader is that they can through our findings find information on the cross-platform approach, and the pure native approach and also through our conclusion see the pros and cons of these approaches.

The study may also contribute to students or other developers who are interested in mobile development. The investigation into popular cross-platform frameworks is very useful and will be of help to many people.

1.3 Scope

The thesis will analyze and compare cross-platform mobile development approaches and therefore contain two frameworks that will be analyzed, the frameworks are Flutter and React-Native. These frameworks will be used to develop two applications towards an android device that will be used to evaluate and to ratiocinate through attributes such as learnability, and performance.

To test the developer's learnability we will aim to complete a specific goal and evaluate the learning curve to succeed. This will be measured from our own experience from three years of Software Engineering studies where we have learned to develop and build software. Where a Software Engineering student achieves basic programming knowledge from C++, Python, Javascript, object-oriented programming, and software architecture courses.

We will also make a survey where we will ask questions regarding different parts in the frameworks to learn what more experienced developers have to say.

The performance will be measured from our two applications that have been built in the respective framework. We will make tests towards each application in terms of CPU usage, execution time, and application size, and then make a conclusion which cross-platform technology is better to use for an application.

1.4 Related Work

Flutter is a relatively new framework, as mentioned above it was initially released in March 2017 and therefore the amount of academic reports is of low existent even if the framework was released nearly three years ago. Unlike from React Native which has been on the market since March 2015, there's a lot of academic reports about comparison towards regular native and reflections regarding resource usage.

Kristiansson published an academic report in 2019 where he compares “Flutter and React Native by code and principle”. He states that for a senior developer it would not be any problem to develop in their respective framework. The major difference in the selected frameworks is that React Native has a bigger community that provides more information and solutions for functionality. But he thinks that in a few years Flutter will probably have a bigger community because it's steadily growing.[14]

Axelsson and Carlström published an academic report in 2016 where they compared “React Native in comparison to mobile native development”. They stated that React Native isn't as good in user experience and performance as the opportunities in mobile native development. They also stated a situational recommendation that React-Native is in an interval of an acceptable range of performance drain to still use React Native if your experience with JavaScript.[15]

Lifh and Lidholm published an academic report 2018, they recreated a native application in the cross-platform framework React Native. The thesis states the differences of performance in react native on the respective platforms Android and iOS. The conclusion of the thesis clearly describes the performance differences where an Android application uses corresponding resources as an android native in such as CPU and memory. Unlike from the iOS which they wrote is much worse than regular iOS native in CPU and Memory usage. The iOS application from React Native used ~50% CPU power corresponding to regular iOS native ~0% CPU power usage.[18]

Further Guzmán and Cernea published an academic report in 2012 where they compared “Cross-platform development of mobile applications” and chose PhoneGap as their framework to evaluate. This thesis stated some problems in the cross-platform approach where it was hard to customize a general graphical view to work on all platforms. There they had a problem with the landscape view to work properly because the Android and IOS kept the resolution and the screen content was just enlarged. Unlike from a windows phone which enlarged the screen content. Their solution to the graphical issue was to add a check function to add specific styling when the user uses the respective mobile platform.[16].

This thesis related work shows clearly that a cross-platform approach is not the best solution from the selected frameworks compared to regular mobile native development. But it's a solution that gives an acceptable range of resource usage and will be less challenging for a senior developer. Cross-platform applications need additional optimization in code to work on all mobile platforms for either graphical or functional use. Solutions are supported for the respective framework, for example, to write customized platform-specific code.^{2 3}

² "Platform Specific Code · React Native." <https://reactnative.dev/docs/platform-specific-code>. Opened 15 may. 2020.

³ "Writing custom platform-specific code - Flutter." <https://flutter.dev/docs/development/platform-integration/platform-channels>. Opened 15 may. 2020.

2. Research Questions

- **RQ - 1 How do Flutter and React Native differ in the development process?**
 - a. How does the debugging process differ between the chosen cross-platform frameworks?
 - b. Which functionalities do the respective cross-platforms support?
 - c. How do flutter and react-native differ in code complexity?
 - d. How do the respective cross-platform applications handle new versions of the operating systems or framework?
- **RQ - 2 How do Flutter and React Native compare in performance?**
 - a. Performance in comparison of:
 - CPU Usage
 - Execution Time
 - Application Size
- **RQ - 3 What are the trade-offs to develop in a cross-platform approach compared to a platform-specific approach?**
 - a. Build a scenario and focus on the following
 - Blueprint app
 - Concept
 - Design
 - Debugging & Testing
 - Release
 - Future updates

2.1 Motivation

The comparisons we want to focus on are mainly the ones which we assume would be important to know about when starting a project using either React Native or Flutter. Another reason behind our research area is because Young Fogelström at Softhouse wanted to know more about this subject, and we believe that these questions will give a proper conclusion.

In RQ1 we want to compare the differences in the frameworks without going in too deep in explaining code or technical dissimilarities. The result we want to get from answering RQ1 is to get an overview of what can be made using the two frameworks and also compare tools when it comes to debugging, external functionalities, and the handling of new versions.

In RQ2 we want to go more in-depth when it comes to the usage of resources that the frameworks need to operate. The tests will be performed under the same standards to be able to make an extensive comparison between the frameworks. Testing the performance of React Native and Flutter will determine which one of the frameworks is the most resource-efficient and can be important for projects which require high performance.

In RQ3 the sub-questions were defined by Young Fogelström at Softhouse because he thought it was an area of interest, to answer if it is beneficial to use a cross-platform framework when making an application suited for both iOS and Android, or if making the application in platform-specific frameworks is less complex. The comparison will be made using a scenario of which an app is being developed from start to finish. The knowledge behind choosing the cross-platform approach or the native approach can be a huge factor when developing a big application. Answering this question will be beneficial for both us, and companies who are in the app industry.

3. Research Method

3.1 Literature Review

This thesis will review other studies to make an accurate summary of the result of similar studies to answer RQ3. The key phrases we will use in our research are, “Native vs Cross-platform”, “Android vs iOS”, “React Native vs Native”, “Flutter vs Native”.

The source of where we will look for related work are:

- Diva Portal
- BTH Summon
- Open Access Theses and Dissertations

When we look for related work we will search for texts which include areas about Native and cross-platform mobile development frameworks. With the help of the texts we find we will investigate the differences between the two techniques when it comes to creating a mobile application.

3.2 Application Experiment

To answer RQ2 we will develop two applications in React Native and Flutter for the android platform, the reason why we only will develop on the Android platform and not the iOS platform is that we lack the required resources to do so. We will then measure the performance using Android Profiler with an android device⁴. Using this tool we will be able to measure several performance parameters to be able to make a clear comparison between the respective frameworks.

The tools we will use to do these performance tests are the following:

- Android Studio⁵ version 3.5.2
- Flutter version 1.12.13
- React-Native version 0.62

The tests measurements we will perform are the following:

- CPU usage
- Execution Time
- Application Size

The tests we will perform are when the application is in an idle state, and once the application is running different functions. We will also monitor the CPU usage when the application is running in the background to measure if there is a big difference in the continuous background running tasks. We will also do the same tests when the application is running in the foreground to see if the test results differ from these two states. The tests will be done multiple times to make sure that we get an average test summary to give a sufficient result.

The tests that will be performed during an active application is:

- An idle background state of the application.
- An idle state of the application.
- Local Notifications.
- Stopwatch/Timer.
- API communication POST/GET.

The tests that will be performed during not active application is:

- Periodic Background.
- Push Notification.

⁴ "Measure app performance with Android Profiler | Android" 14 jan.. 2020, [Measure app performance with Android Profiler..](#)

⁵ "Android Studio - Android Developers." <https://developer.android.com/studio>. Opened 14 apr.. 2020.

3.3 Mobile Application Concept & Code Variation

To answer RQ1 we will research the external functionalities and tools used by the frameworks. Therefore, as stated above, in this thesis two mobile applications will be created. We collaborated with an employee at a software company about what features should be included and which aspects are interesting to dive deeper into. The features the employee said is most interesting and wanted us to dive deeper into was:

- Periodic background running
- Sensors / GPS
- Notifications (Push)
- Communication / REST API

3.3.1 Application Idea

The basic idea we got from the collected information was to make an application where the user creates activities and then takes time on how long the activities take. For example, if the user is going on a run, the user then makes an activity named “Running” and starts the timer, then when the activity is completed the user stops the timer and it will be saved to a list of completed activities.

Using this idea we could use periodic background running when the user has started an activity and the timer is activated. We could also use GPS to track the user’s current location when an activity is started/stopped. The use of notification could also be used here to notify the user for example when an activity has been run for a certain amount of time to remind the user that an activity is active. And to synchronize the user with the activities and to add/remove activities we will use a REST API to communicate with a database to save all the data.

3.3.2 Main Application Functionality

- Periodic background is used to notify the user every 15 minutes through push notifications.
- A GPS point is fetched when an activity is created to track the user’s current location.
- A timer is implemented to calculate up each second the activity is active both in background state and active state.
- ArangoDB⁶ REST API is used to store unique device id in the database as a user key.
- ArangoDB REST API is used to create/remove/update/delete activities in the app.

To answer RQ1 about how React Native and Flutter differ when it comes to development we will do a few comparisons when it comes to the variation of developing code in the different frameworks.

⁶ "ArangoDB." <https://www.arangodb.com/>. Opened 20 april.. 2020.

3.3.2.1 Framework Packages

The listed packages will be used in the respective framework as a method to support the test functionality identified above.

3.3.2.1.1 React-Native

- Periodic background running

The periodic background will be run by a package called react-native-background-fetch that works both in an iOS and Android environment⁷. This package triggers a simple background-fetch event every 15 minutes that provides a short background execution-time which triggers your customized callbackFn.

- Sensor / Global Positioning System - GPS

The GPS point will be fetched from a package called react-native-geolocation that works both in iOS and Android environment⁸.

- Device information

The device information is will be run by a package called react-native-device-info that works both in iOS and Android environments. This package gives you around 67 options to fetch different information from a device running your application. In this thesis we are only interested in each unique id of a device.⁹

- Push Notifications

The notifications will be managed through a package called react-native-push-notifications which supports both local notifications and remote notifications in iOS and Android.¹⁰

- Communication / REST API

The communication between the ArangoDB and the react-native application will be handled with HTTP requests where react-native provides you the option to use a Fetch API¹¹.

⁷ "Background-Fetch | React-Native Package" <https://github.com/transistorsoft/react-native-background-fetch> Opened 20th of april.. 2020.

⁸"Geolocation | React-Native Package" <https://github.com/react-native-community/react-native-geolocation> Opened 6th of april..2020

⁹ "react-native-device-info - npm." 5 may. 2020, <https://www.npmjs.com/package/react-native-device-info>. Opened 15th may. 2020.

¹⁰ "zo0r/react-native-push-notification: React Native ... - GitHub." <https://github.com/zo0r/react-native-push-notification>. Opened 30 april. 2020.

¹¹"Fetch API | React-Native" <https://reactnative.dev/docs/network> Opened 7 april.2020

3.3.2.1.2 Flutter

- Periodic background running
The periodic background running in Flutter is made by using the package WorkManager¹². WorkManager is a wrapper around Android's WorkManager which effectively enables the headless execution of code in the background.
- Sensors / GPS
To retrieve the current location of the user with the Flutter application the package Geolocator¹³ was used. This package has a feature to get the current location of the device which is exactly what we wanted.
- Device information
To get the current device information the package device_info¹⁴ was used. This package provides detailed information about the device which we needed to link the activities to the unique device ID.
- Notifications (Push)
Using the package WorkManager as mentioned above, it is possible to schedule notifications. However the package does not allow a notification frequency under 15 minutes.
- Communication / REST API
The communication between the ArangoDB and the flutter application was handled with HTTP requests using the package http¹⁵. The http package made it easy to use all types of HTTP requests to get/update/delete/add information to the database.

3.3.3 Debugging

We will compare the debugging tools we used during our development of the applications and describe our thoughts on the debugging process, to answer the RQ1 about the difference when it comes to debugging functionalities of the frameworks.

¹² "workmanager | Flutter Package - Dart Pub." 5 mars. 2020, <https://pub.dev/packages/workmanager>. Opened 20 apr.. 2020.

¹³ "geolocator | Flutter Package - Dart packages." 1 apr.. 2020, <https://pub.dev/packages/geolocator>. Opened 15 may. 2020.

¹⁴ "device_info | Flutter Package - Dart Pub." 11 mars. 2020, https://pub.dev/packages/device_info. Opened 20 apr.. 2020.

¹⁵ "http | Dart Package - Dart Pub." 7 jan.. 2020, <https://pub.dev/packages/http>. Opened 20 apr.. 2020.

3.3.4 Code Complexity

As stated in RQ1, we want to compare how the code complexity differs between React Native and Flutter. We will compare functions in the application and compare both the difficulty of importing these functions in terms of lines of code, and also in terms of own experience if we thought it was hard or easy to implement.

To avoid faulty LoC(lines of code) size differences we will exclude comments and make sure that code with a single opening bracket is excluded as well.

For example and "If" statement will look like this:

- ```
if(true){
 Code
}
```

And not like:

- ```
if(true)  
{  
  Code  
}
```

3.3.5 Versions

New versions of a framework can be dangerous for a project sometimes, a new update might cause a lot of problems and it can sometimes be advised to not update to a new version just because of this. We want to compare how updating to a new version can impact our applications to see if the frameworks handle new versions to a good extent, or if it's a problem.

3.4 Survey

To answer RQ1, a survey with questions about React Native and Flutter will be made to help in reaching a conclusion. The target audience we want to reach out to is both inexperienced developers and more experienced developers to be able to get a wide result from both groups. To be able to reach both the target groups, we put out our survey on Reddit¹⁶, for both the subreddits for React Native, and Flutter.

The result from creating our own apps and the answers we receive from the survey will be used to answer our research question with a more broad perspective.

The survey will include questions regarding how many years of experience the respondent has within React Native or Flutter, with this info it is possible to see if the results differ a lot between an experienced developer or a beginner.

We will also ask questions about debugging, if the respondent thinks that the built-in tools are good enough or if they believe that external debugging tools are required. And if so, what external debugging tools they used. Both React Native and Flutter rely on third-party external packages to import functionality, so we wanted to know if the respondents thought that this was a good method and why.

And finally, we want to know if the respondents think that there is something missing to the framework and if they would recommend learning this technology to someone who has never used it before.

To analyze the survey we will look at the answers which the respondents provide and see how the different answers differs from a more experienced respondent to a less experienced. We will then divide these into two groups and conclude the results between them.

¹⁶ "Reddit – Wikipedia." <https://sv.wikipedia.org/wiki/Reddit>. Öppnades 5 juni. 2020.

4. Literature Review

There are many studies made on cross-platform mobile frameworks and native frameworks. The ones we will review in our literature review are [8][9][10][12][13][16][17]. To do the review we will focus on a scenario standpoint where we will emphasize all the steps when creating an application, from start to end.

During a mobile application project, there are a few development stages which the developer always will go through. The first step is to create a blueprint app model to decide what the application should approximately look like. Then the concept of the application will be worked on to know what the purpose of the app is. Once this is done then the design of the app should be decided so that it is clear once the development starts for the developers to know how to implement the design. During the implementation, debugging and testing must be done to make sure that the app is robust and will handle the functions which have been given. And finally the app will be released on the market.

This study will focus on these areas and compare how much it differs when it comes to a cross-platform framework and a native framework.

4.1 Blueprint App

The application blueprint is very much what it sounds like, a model of what an application should look like. When it comes to creating a blueprint for an application whether it be a native or a cross-platform application, the blueprint will not differ too much because of functionality-wise. The most obvious difference is the design, it will vary a little bit on the native frameworks depending on the OS, for example Android has a built in “back button” which iOS does not have. As stated in [13], the blueprint app is important when it comes to be able to receive changes that the customers want and incrementally build the new functionality after each iteration.

4.2 Concept

There are a few points that need to be considered when it comes to creating the concept app, but many of these depend on what type of application is being developed. As stated in the conclusion from [8], developing in cross-platform is a good choice when developing simple apps. Cross-platform can save a lot of time because it can develop one app that works on all the platforms. But if you want to get the most responsive application it is better to choose the Native route.

4.3 Design

As stated in the conclusion from [8] and [10] an app developed in a cross-platform framework looks very similar in comparison to the native apps. The native frameworks will have some OS-specific design details which will differ from the cross-platform design. The design created in cross-platform means that the UI is built once for all platforms which makes the design consistent across platforms.

4.4 Debugging & Testing

The debugging and testing process in either native or cross-platform is, for the most part, one streamlined process regardless of different target platforms, as stated in [9]. According to their thesis, the process is usually to build and test it on different browsers and then port the same code to different platforms to build for those platforms, which saves time and money. When we did our survey however the result we got was that the most common test strategy was to use console commands, which is also the method we used when we tested the applications we created. The testing becomes easier and faster as both platforms have quite the same behavior. This means that logical bugs won't be unique per platform and finding and fixing it on one platform will be reflected to another, as stated in [12].

4.5 Release

There are a few differences when it comes to releasing an application made in a cross-platform framework compared to a native framework. A released native application will be faster, more responsive and more energy conservative compared to cross-platform, but the cross-platform will much faster be able to get on the market because the cross-platform approach will immediately be available for both OS. As mentioned in [16], the native app is, for the most part, faster and smoother, but if you want to quickly develop and are willing to compromise the design and performance, a cross-platform approach can be very useful and economical. Another factor is also that an app made in a cross-platform framework requires approximately the same amount of code required from a native app. So if the app should target both iOS and Android, going the native route requires double the amount of coding, one app in Android Native, and one app in iOS native, as mentioned in [17].

Releasing the application which targets both OS will have a much broader audience, which increases the chances of success of the app.

4.6 Future updates in Flutter & React Native

Generally, when it comes to updating a cross-platform project, there are difficulties when it comes to upgrading the framework to a newer version. As stated in [9], many updates in cross-platform can be done automatically which will guarantee the latest version to the users. This sounds good on paper, but in reality, upgrading something to the latest version can sometimes cause problems in other areas of the application. For example upgrading a functionality within the app could cause integration issues between other functions. And because cross-platform targets both iOS and Android, the risk of version mismatch is even higher. Because of this reason, it is safer to develop in a cross-platform when creating simple apps with a short life period.

5. Results

5.1 Application Experiment

5.1.1 Environment Installation

5.1.1.1 Flutter

The installation of Flutter was very easy. First, you need to download an SDK that contains the latest version of flutter. Then you need to add Flutter to your PATH environment variable if you are on Windows, to get access to the flutter commands in the terminal. Then you run “flutter doctor” in the terminal to check if there are any missing dependencies. The dependencies are:

- Newest Flutter version
- Android toolchain
- Android Studio
- Connected Devices

After the command has been run, it will then give information about how to install these if there are any.

5.1.1.2 React Native

To install React Native there were plenty of guides showing how to install, but the one followed were the official react-native getting started guide.¹⁷ Which showed guides for common operating systems like Mac OS, Windows, and Linux. The chosen operating system for this thesis application was Linux(Ubuntu 18) and the React-Native installation went very smoothly. The only complex in the installation is the android studio to install the right packages and configure the emulator to run.

Although Flutter was installed on Windows, and React Native was installed on Linux this will have no impact on the performance tests we will perform because the app will be run on the same mobile device.

The initial stage to install React-Native where to get the Java Development Kit (JDK), then the installation continued with Android Studio to make a customized installation to guarantee the Android SDK, Android SDK Platform and Android Virtual Device was included.

¹⁷ "Setting up the development environment" <https://reactnative.dev/docs/environment-setup> Opened 5 march..2020

The second stage is to install Android Pie (9) and three additional Android Pie (9) packages: Android SDK Platform 28, Intel x86 Atom_64 System Image, or Google APIs Intel x86 Atom System Image.

The third stage were to add the following environment variables lines to your \$HOME/.bash_profile or \$HOME/.bashrc config file:

```
export ANDROID_HOME=$HOME/Android/Sdk
export PATH=$PATH:$ANDROID_HOME/emulator
export PATH=$PATH:$ANDROID_HOME/tools
export PATH=$PATH:$ANDROID_HOME/tools/bin
export PATH=$PATH:$ANDROID_HOME/platform-tools
```

The last stage of the installation was to create a project through `npx react-native init <project name>` and to run `npx react-native run-android` which installed the react-native application to the emulator and explained in the getting started guide also start the emulator with a running local server.

There was a bug in the react-native 0.62 where the local server host closed after a successful installation. To solve this you had to write an `npx react-native start` and then when the server started you could write `npx react-native run-android`. This made sure that the local server still was running and you could develop the application.

5.2 Packages

5.2.1 Flutter

The packages used in the application was the following:

- device_info
- http
- geolocator
- workmanager

5.2.2 React Native

React-native packages used to develop the application functionality.

- react-native device-info.
- react-native background-timer.¹⁸
- react-native geolocation.
- react-native background-fetch.
- react-native push-notifications.

¹⁸ "react-native-background-timer - npm." Opened 10 march. 2020, <https://www.npmjs.com/package/react-native-background-timer>. Opened 23 apr.. 2020.

5.2.3 Package Installation

5.2.3.1 Flutter

To install a package, first, the package name must be added to the pubspec.yaml file under “dependencies”, then run “flutter pub get” in the terminal to update your project with the new package. If the application is being run in Android Studio, it will handle it for the developer, just simply click “update dependencies” in the top of the screen, it will detect if a new package is added automatically.

5.2.3.2 React Native

The installed react-native version was 0.62 which led to an easier package installation process because this version supports automatically package links. When installing a package you only have to write “npm install “package-name” --save”.

5.3 Contents

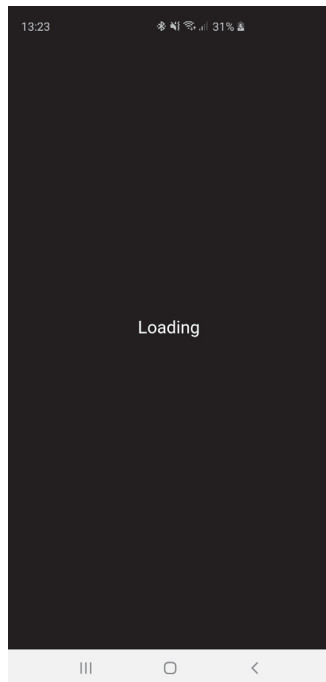
5.3.1 Flutter

The application was made using 3 screens.

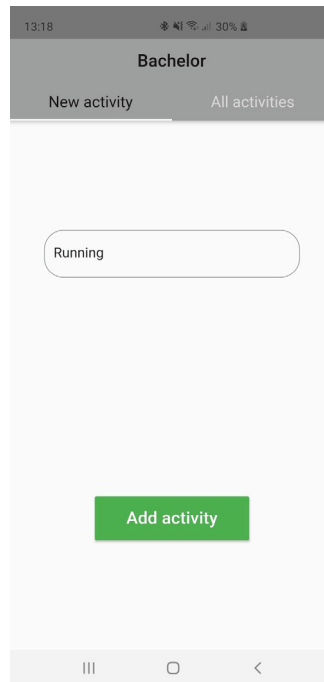
The StartScreen will just do some background checks to see if the user is already saved in the database, or if the user must be added.

The HomeScreen will be the main “hub” of the application where the user can create new activities or monitor already saved activities. The user can also remove a selected activity.

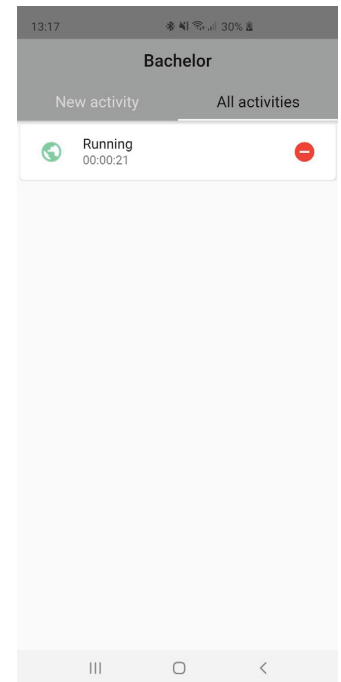
The StopwatchScreen is where the user can start/stop/reset an activity through buttons displayed on the screen. The user can then choose either to save the activity or to cancel the activity.



Start Screen

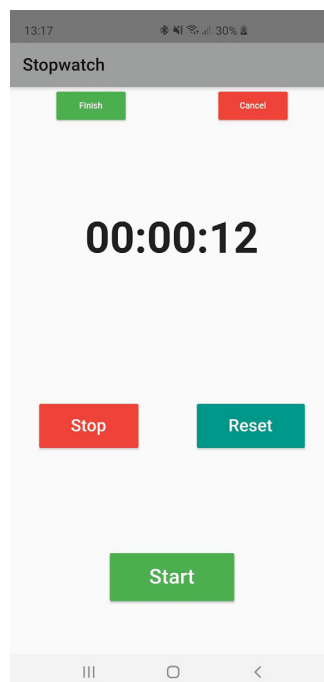


Tab 1



Tab 2

Home Screen

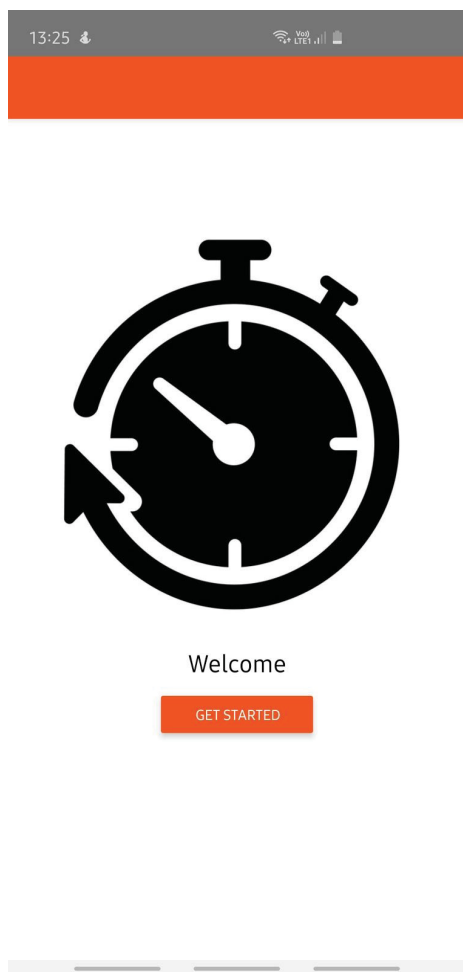


Stopwatch Screen

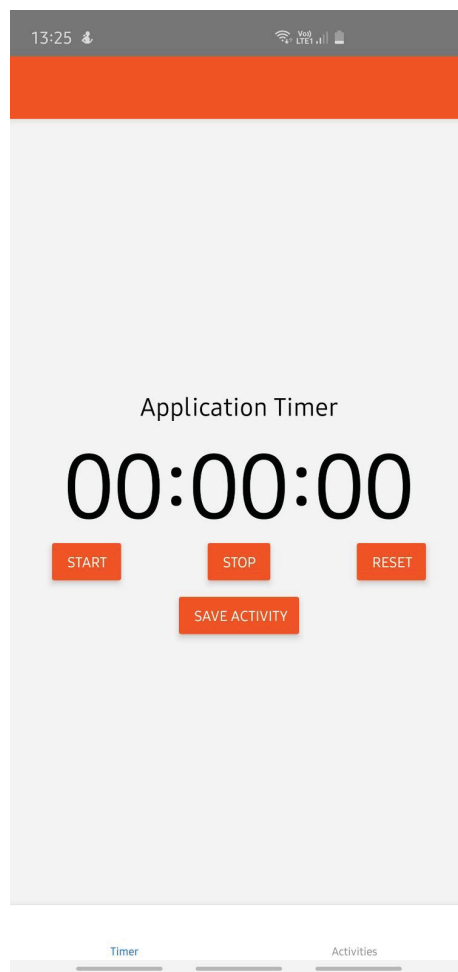
5.3.2 React Native

The react-native application is made with two screens, the authentication, and the home screen. An authentication screen is only made to make sure the user is registered in the system and the home screen is the main part of the functionality which contains a tab view with two view options as timer view and activity view. In the timer view, you can start, stop, reset, save the activity and the activity view shows the saved activities.

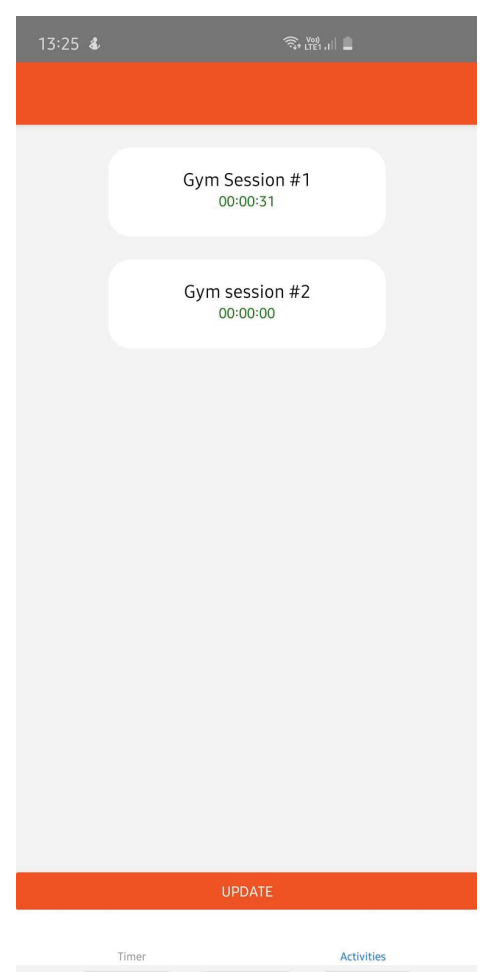
The application has a user interaction through push notifications where notification is present when an activity is started and a notification running as periodic background when the app is killed to inform the user to not forget to actively register activities.



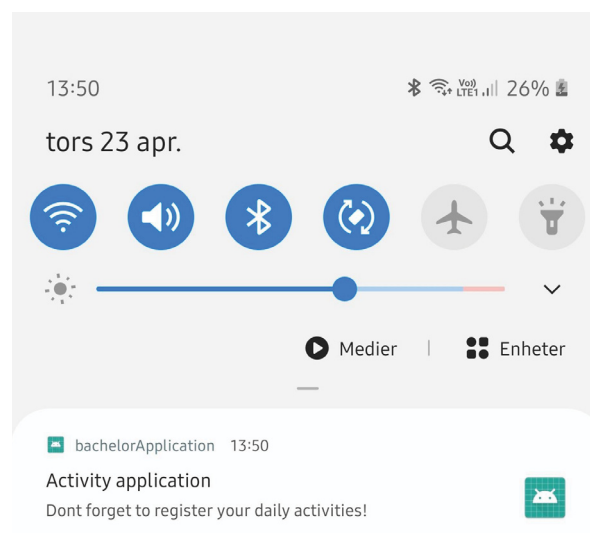
Authentication



Home "Tab View Timer"



Home "Tab View Activities"



Push notification

5.4 Functionality

We initially wanted to include GPS functionality in the application, but during the development we found out that in order to use Google Maps API on Flutter it is required to choose a payment method. That is why we did not include this in our experiment.

5.4.1 Flutter

StartScreen:

First, when the application is run, it will go through the “Start Screen”. This screen will only show a text saying “Loading” to the user, while in the background it will check if the current user is already in the database, or if it is a new use that needs to be saved. Then it will do a check to see if the user currently has any activities that have been started. If there is an activity started then the application will immediately direct the user to the stopwatch page.

HomeScreen:

The “Home Screen” page is where the user can either start a new activity by entering the name of the activity that will be created. Or the user can show all completed activities by navigating through a tab at the top of the screen. On the activity tab, the user can see all the recent activities and also remove a selected activity. If the user wants to create a new activity a name has to be given to this activity, then once the user is done the application will navigate to the “Stopwatch Screen”.

StopwatchScreen:

On the “Stopwatch Screen” there will be 5 buttons to control the stopwatch. Each button has its own functionality to control the stopwatch, the buttons which are included are, start button, stop button, reset button, cancel button, and finish button. When the start button is pressed the first thing that happens is that it will create an activity and post it to the database. Then it will start a background task process to notify the user every 15 minutes. And finally the stopwatch will start ticking. If the user presses the stop button what happens is that the current activity will update to the database and indicate that it has stopped, then it will stop the stopwatch. The reset button will update the current activity indicating that it has been reset, then it will reset the stopwatch. The cancel button will remove the current activity from the database, reset the stopwatch, remove the background task, and finally navigate the user to the “Home Screen”. And finally the finish button will stop the stopwatch, save the activity to the database with the stopwatch time, remove the background task, and then navigate the user to the “Home Screen”.

5.4.2 React Native

The react-native fetch API functionality is implemented in the get started, save activity, and update activities button. Get started button makes a regular get request towards the API to check if the user is registered in the system, if not the user is added through a POST request. Save activity saves the current activity in the database through a POST request and the update button makes a GET request to update the activity list.

The user is saved in the database through the user device unique id and the application will inform the user about specific information in local push notifications. A notification will inform the user to not forget to actively track your activities every 15 minutes when the app is killed as a periodic background running and a notification when the timer activity is started.

The start timer is implemented with javascript setInterval() with an interval of one second where a react-native setState() operation is called each second to calculate up a variable to represent the time, stop and reset is javascript clearInterval(). The clearInterval removes the setInterval() and when reset is pressed a react-native setState() is activated to reset the timer variable to zero. To enable the application timer in the foreground the react-native package “react-native-background-timer” is being used which extends the javascript interval functionality.¹⁹

¹⁹ "react-native-background-timer - npm." 10 march. 2020,
<https://www.npmjs.com/package/react-native-background-timer>. Opened 23 apr.. 2020.

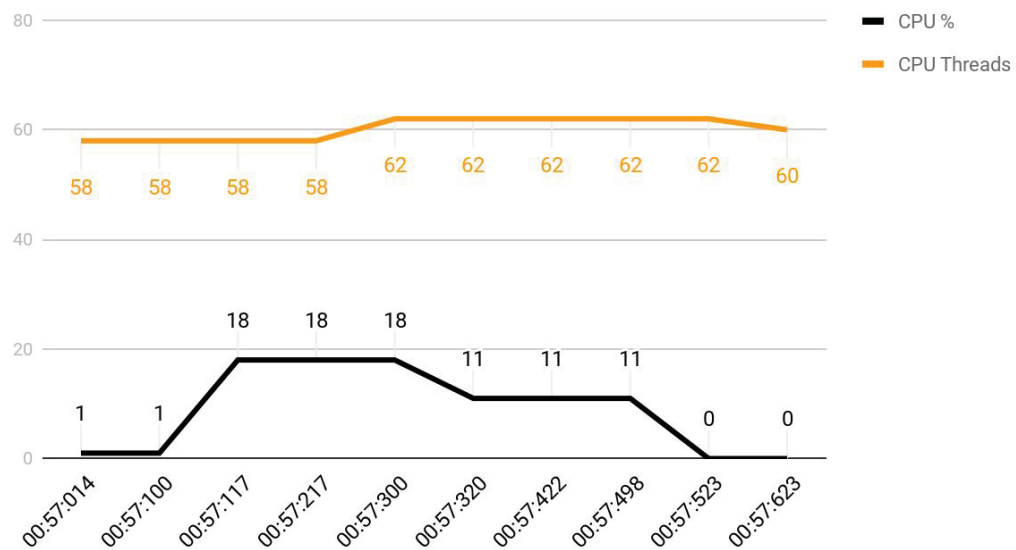
5.5 Performance Tests

5.5.1 CPU Tests

5.5.1.1 Flutter

These Flutter performance tests were done on the profiler functionality in Android Studio with a real Samsung Galaxy S10+ device connected.

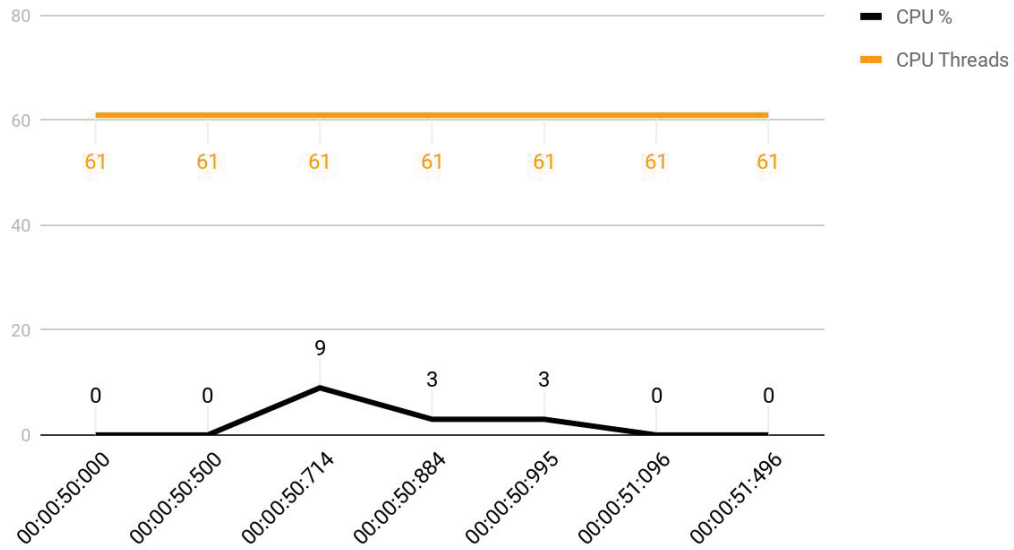
Flutter - Http "POST"



This test visualizes the CPU usage of the flutter application that uses the Http "POST" to create an activity, initiate background tasks, and start the timer.

- Max: 18%
- Average: 14.5%
- Min: 11%

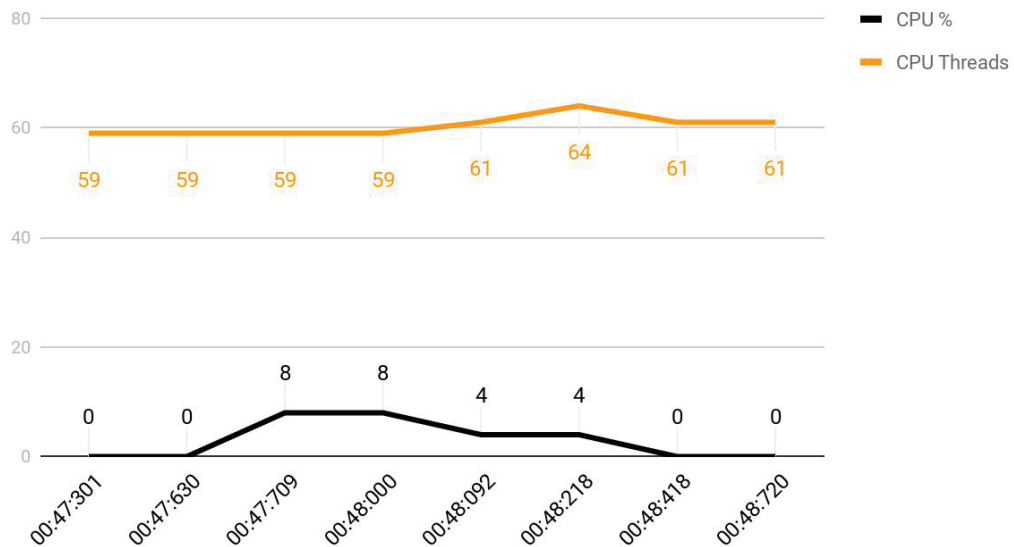
Flutter Http "GET"



This test visualizes the CPU usage of the flutter application that uses the Http "GET" to fetch all activities.

- Max: 9%
- Average: 5%
- Min: 3%

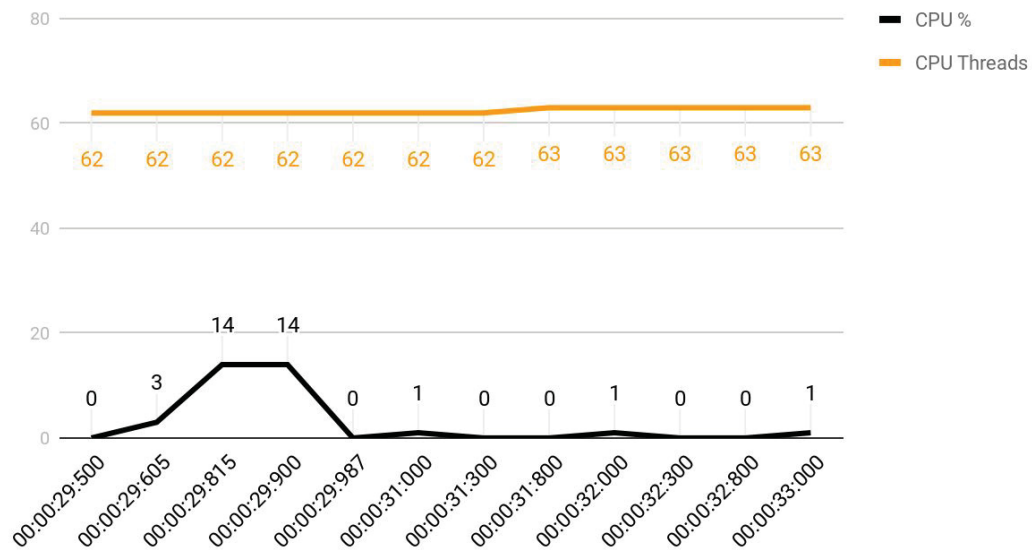
Flutter - local notification



This test visualizes the CPU usage of the flutter application that uses the workmanager package to send a notification.

- Max: 8%
- Average: 6%
- Min: 4%

Flutter - stopWatch



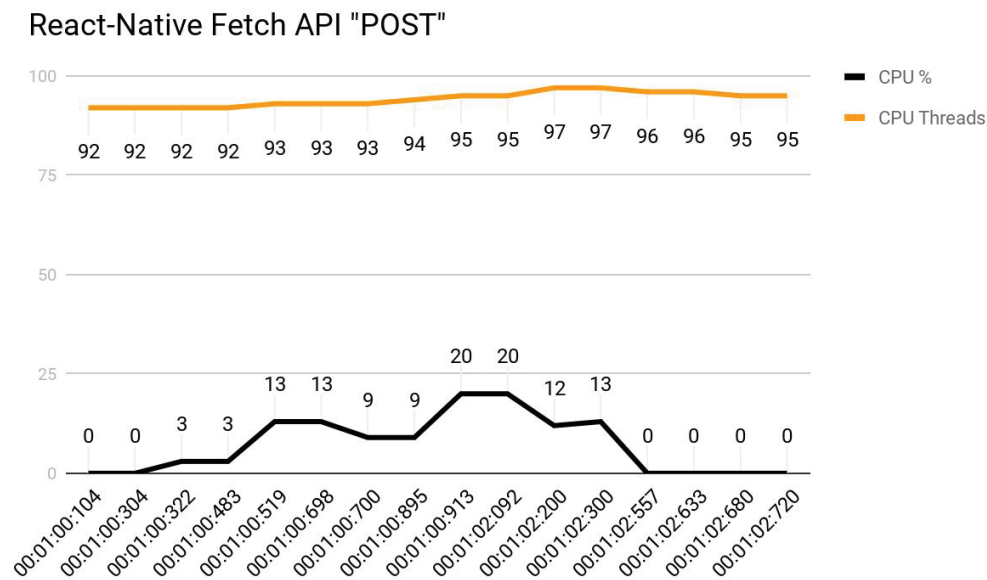
This test visualizes the CPU usage of the flutter application when the stopwatch is started and then CPU usage when the stopwatch is running.

- Max: 1%
- Average: 0,375%
- Min: 0%

The tests were performed under normal conditions and when the function to register an activity the CPU usage went up to 18% as its highest. When the function to fetch activity information was called the CPU usage went up to 9% as its highest. And when the notification was called the CPU usage was at its highest 8%. When the stopwatch was started the CPU usage went up to 14% because an activity is posted to the database, it then only uses 0-1% CPU usage when the stopwatch is active.

5.5.1.2 React Native

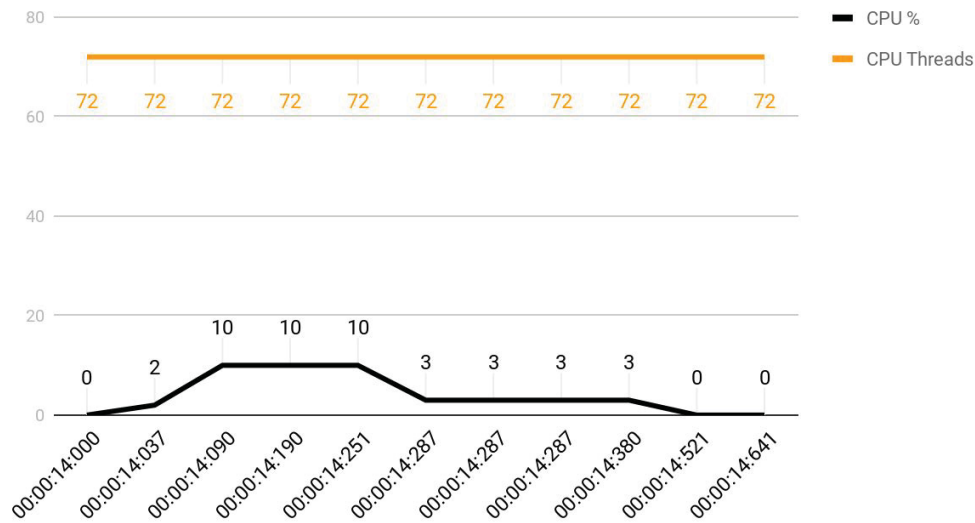
These React Native performance tests were done on the profiler functionality in Android Studio with a real Samsung Galaxy S10+ device connected.



This test visualizes the CPU usage of the react-native application that uses the Fetch API "POST" when an activity is published.

- Max: 20%
- Average: 11.5%
- Min: 3%

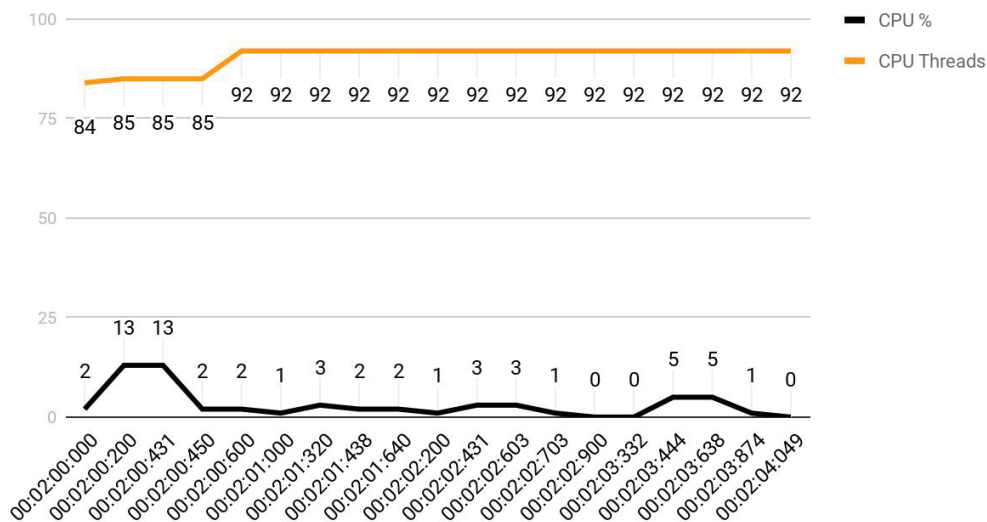
React-Native Fetch API "GET"



This test visualizes the CPU usage of the react-native application that uses the Fetch API "GET" to fetch all user activities

- Max: 10%
- Average: 5.5%
- Min: 2%

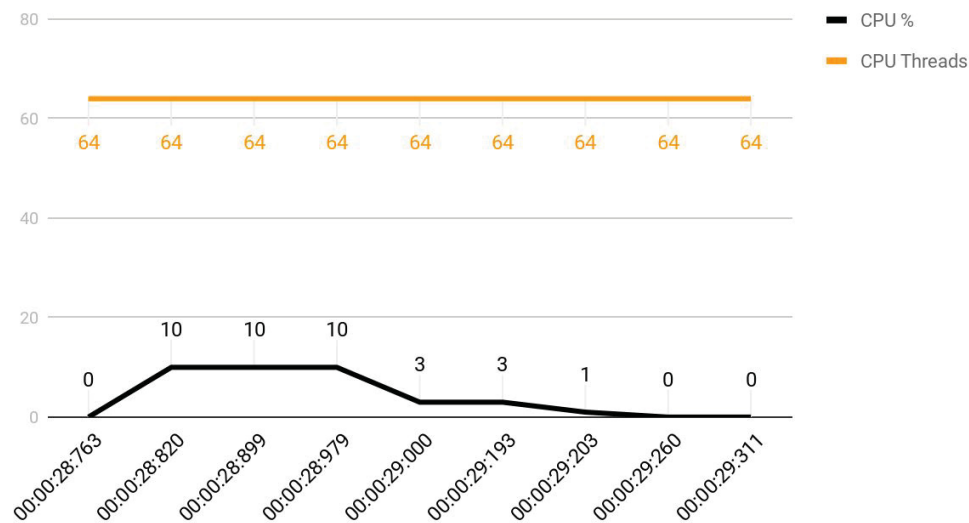
React-Native-Background-Timer



This test visualizes the CPU usage of the react-native application to calculate the timer to start with javascript setInterval() and calculate further with react-native setState() each second.

- Max: 3%
- Average: 2.07%
- Min: 1%

React-Native Push-Notification "Local Notification"



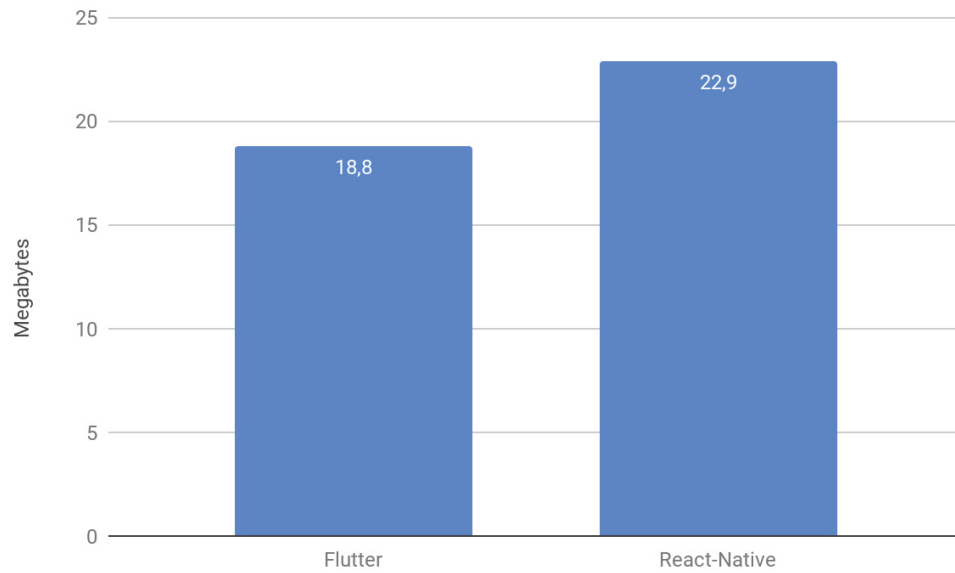
This test visualizes the CPU usage of the react-native push notification to notify the user through a local notification

- Max: 10%
- Average: 6.16%
- Min: 1%

The application was monitored under regular conditions, where the user used the application to register their activities. Idle foreground, active with no user interaction, and active with user interaction were monitored to give a curve of average CPU power used. When the application remained in the idle foreground or was active without user interaction the app drained on average 0 to 1 percentage CPU power. Then when user interaction started to interact either with the navigation or usage of the screen an interval of 5 to 8 percentage of CPU power was consumed.

The 15 minutes of periodic notification could not be tested in the React-Native application because Android Studio only supported the profiling of an active application and the periodic background is activated when the app is killed.

5.6 Application Size



The final application size that was generated as release APK:s is Flutter at 18,8 MB and React-Native at 22.7 MB..

5.7 Code Complexity

To compare the code complexity we have chosen a few functions in our application and will present code & reasoning behind implementing them. We will then come to a conclusion on which framework is more complex when it comes to implementing these functions.

5.7.1 React-Native Periodic Background Running

```
let headlessTask = async (event) => {
  let taskId = event.taskId;

  PushNotification.localNotification({
    title: "Activity application",
    message: "Don't forget to register your daily activities!",
    playSound: true,
    soundName: "default",
  });
  BackgroundFetch.finish(taskId);
}
BackgroundFetch.registerHeadlessTask(headlessTask);
```

A headlessTask(periodic background task) is initiated that has default 15 min interval configuration and the push notification package is used to locally notify the user.

5.7.2 React-Native Push-Notification

```
PushNotification.localNotification({
  title: "Activity application",
  message: "Don't forget to register your daily activities!",
  playSound: true,
  soundName: "default",
});
```

Push a local notification directly to the phone.

This package allows 25+ of configuration options on an notification, in this application title(Notification title), message(Notification message), playSound and soundName were used.

5.7.3 React-Native Fetch API Communication

```
let response = await fetch('http://' + this.state.database.ip +
  '/_db/Bachelor/activities_crud/activities', {
  method: 'POST',
  headers: {
    Accept: 'application/json',
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    user: this.state.uniqueId,
    activity: activityName,
    time: this.timeConvert(),
    date: date,
    lat: longitude,
    long: latitude,
    status: "Finished Activity"
  }),
});

let response = fetch('http://' + this.state.database.ip +
  '/_db/Bachelor/activities_crud/activities', {
  method: "GET",
  headers: {
    'Accept': 'application/json',
    'Content-Type': 'application/json',
  },
});
```

POST & GET towards API.

In the post function, the headers and body must be set to be able to send data

In the get function its possible to act according to http status codes with “response.status”,
and then handle the response through JSON

5.7.4 Flutter Periodic Background Running

```
Workmanager.initialize(  
    callbackDispatcher,  
    isInDebugMode: true  
);  
Workmanager.registerPeriodicTask(  
    "1",  
    "Duration",  
    frequency: Duration(minutes: 15),  
);  
Workmanager.executeTask((task, inputData) {  
    return Future.value(true);  
});
```

Initialize and register a background task and send notification

Flutter only allows a notification frequency of minimum 15 minutes which is configured under “frequency”

5.7.5 Flutter API Communication

```
final response = await http.post("http://" + ip['ip'] +  
    "/_db/Bachelor/user_crud/users",  
    headers: <String, String>{  
        'Content-Type': 'application/json; charset=UTF-8',  
    },  
    body: jsonEncode(<String, String>{  
        '_key': id  
    })),  
);  
  
final response = await http.get("http://" + ip['ip'] +  
    "/_db/Bachelor/activities_crud/activities");  
if (response.statusCode == 200) {  
    var data = json.decode(response.body);  
}
```

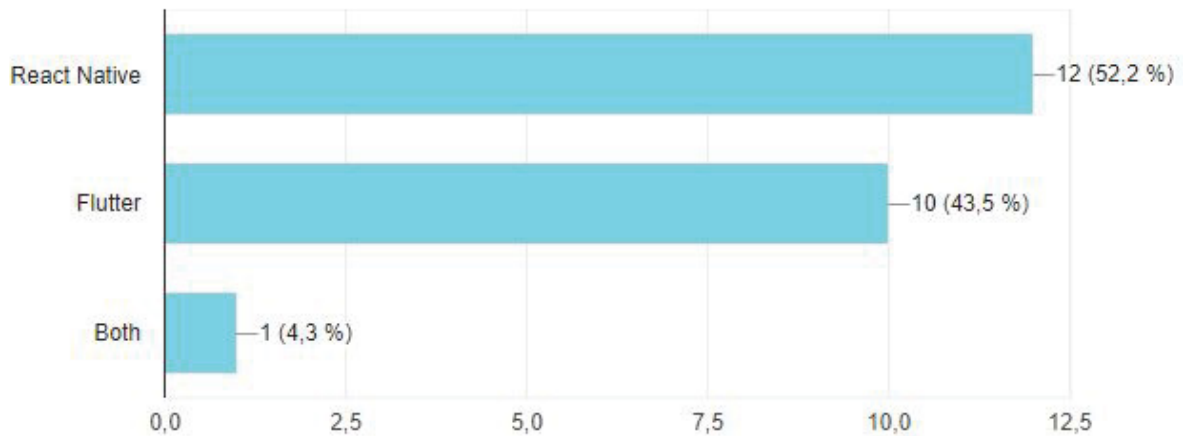
POST & GET towards API

In the post function, the headers and body must be set to be able to send data

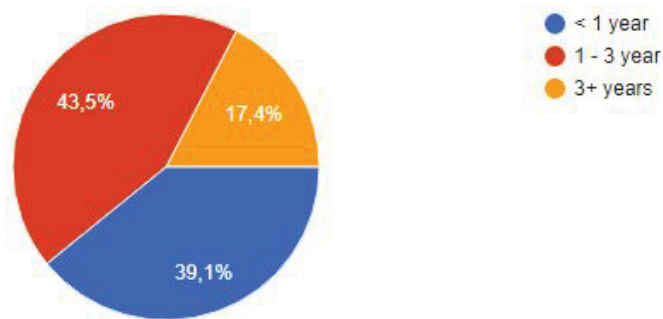
In the get function its possible to act according to http status codes, and then handle the response through JSON

5.8 Survey Results

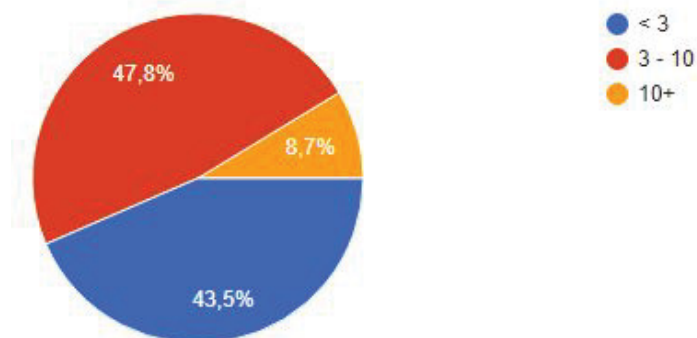
Those who participated in our survey started by filling in information on what framework they are most familiar with, either Flutter or React Native. We then wanted to know how long the respondents have been working with the chosen technology, and also how many projects the respondent has done. With this information, we could conclude how experience within the framework will affect the answers. All the survey questions and answers can be found down in annexes. Here are the questions regarding the respondents experience.



Respondents familiarity with the frameworks



Respondents experience with the chosen framework



Number of projects done with the chosen framework

6. Experiment Analysis & Reflection

6.1 Development

RQ1 conclusion

6.1.1 Code Complexity

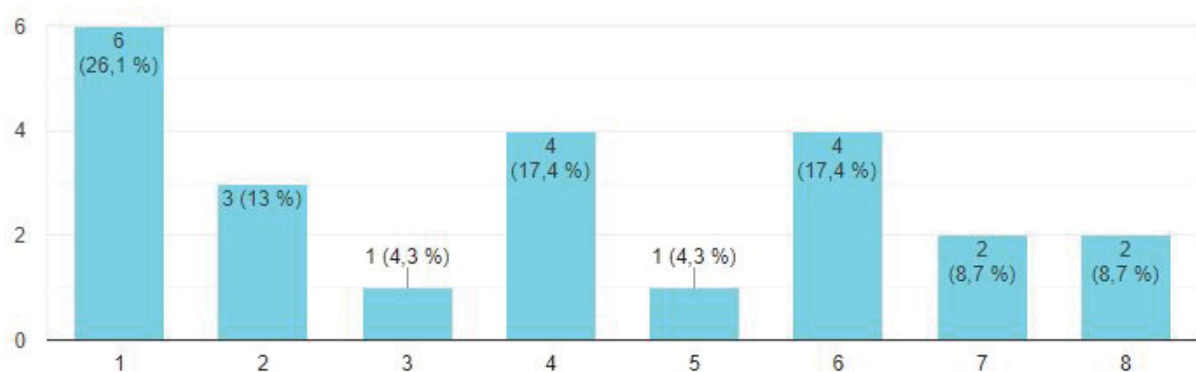
In chapter 5.7, code examples of respective main functionality were posted from Dart (Flutter) and Javascript (React Native). The complexity of implementing functionality in Flutter and React Native is fairly the same and in this thesis, they mainly differed in the selected external packages. In Flutter the selected external packages did not require any complex configuration and were quick to set up. Unlike for the selected external packages to React Native in which there were a lot of configurations to get the expected functionality.

6.1.2 Debugging

Survey Result:

From the survey result, most people think that debugging using built-in tools is hard. Though the people who thought the debugging was hard were mostly respondents who had less than one year of experience.

One question in the survey was about what debugging tools the respondents use when debugging and the most used technique was to print our logs in the console for both React Native and Flutter. However, this technique was mostly used by less experienced respondents. React Native respondents with more experience used tools like, React Native Debugger, Flipper. While less experienced respondents used console logs and Stackoverflow. Flutter respondents with more experience used tools like Flutter Inspector, while less experienced respondents used console logs.



Survey question about how easy it is to debug a project with built-in tools with difficulty scale 1-8

Our Experience:

When creating the Flutter and the React Native application, the debugging tools used was to print comments in the console. This method was sufficient when developing our application and looking at the survey result, the less experienced respondents used console commands so the conclusion from the survey fit in our conclusion as well.

6.1.3 Supported External Functionalities

Survey Result:

Both React Native and Flutter use external packages to import different functionalities to the project. We asked our respondents if they think this method is good, or if they prefer all functionalities to already be in the “base” of the framework. Everyone doing the survey thought that this method is good, mostly because the community for both the frameworks is very driven and sometimes the community-driven packages are better to use than the internal modules.

Another strong point that we got from the survey was that it is sometimes better for the developer to pick the best tool for each use case whether it be an external package or an internal module. The frameworks only deliver the bare minimum and give you freedom of choice, which is a good way for the developer to have a bigger decision in the project.

Our Experience:

When developing the Flutter and React Native applications a few external packages were installed, without them, the application could not have been made. Installing external packages is a very good way to give the developer exactly what is needed in order to make a certain functionality within the application. It is very easy to install a package and each package has a good description of how the package is used. Because of new packages being made it means that the evolution of the framework can be improved a lot in the future, new functionalities can be added and improved using this method.

6.1.5 Version Handling

6.1.5.1 Flutter

To update both the Flutter SDK and the packages that the application depends on there is a command which can be run from the root of the app which will do this automatically²⁰. The command is “flutter upgrade”, and it will update each package to the most recent compatible version. It is also possible to upgrade the packages only instead of both packages and Flutter itself. To do this the command is “flutter pub get” and then “flutter pub upgrade”. It is also possible to select a specific version when upgrading with the “flutter version” command.

²⁰ "Upgrading Flutter - Flutter." <https://flutter.dev/docs/development/tools/sdk/upgrading>. Opened 30 apr.. 2020.

6.1.5.2 React Native

To upgrade React Native projects there's currently two ways, either by using React Native CLI²¹, or manually with Upgrade Helper²².

The React Native CLI has a command "react-native upgrade" which will start the process of upgrading the project to the latest version. There is also a possibility to upgrade to a specific version by passing an argument after the "upgrade" command to specify what version is desired.

The Upgrade Helper is a tool which you can use to get a detailed description what is needed to be updated in the code in order to perform an upgrade. Using this tool you first need to specify the current version of your project and the targeted version you want to upgrade to. Then you will be shown a guide on what files need to be changed in order to upgrade to the newest version.

To upgrade all the package dependencies in a project it is possible to do this using npm²³. There is a command which will check if there are any new version do all the dependencies which is "npm-check-updates". Writing "npm i -g npm-check-updates", will install all the newest dependencies in the current project.

6.1.6 Summary

Both Flutter and React Native are similar functionality wise, they both use external packages which are easily installed and implemented. Debugging and testing are fairly similar as well, there are some framework-specific debugging tools that can be used, but other than that the process is the same. Code complexity is mostly decided on external packages. Some packages are simple and some require more configuration. Both Flutter and React Native have built-in tools to help with version control. Flutter uses console commands which can be configured to update specific versions or automatically update the entire project. React Native has a similar function, but also an approach to manually upgrade the packages, this could be very helpful if there is a package which has high coupling in the project.

²¹ "react-native-cli - npm." <https://www.npmjs.com/package/react-native-cli>. Opened 30 apr.. 2020.

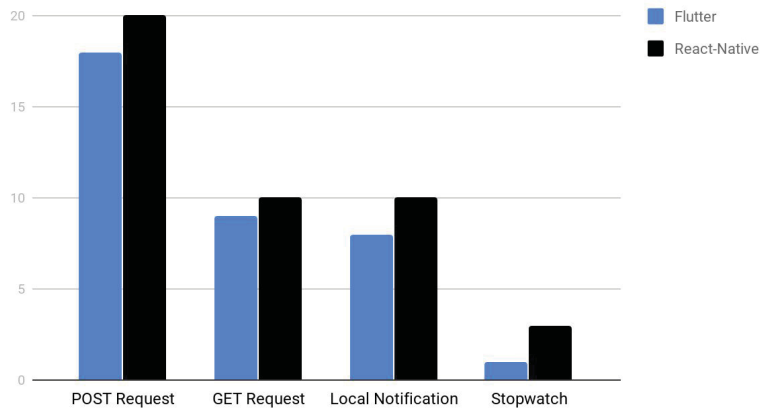
²² "Upgrade Helper - Upgrade React Native applications." <https://react-native-community.github.io/upgrade-helper/>. Opened 30 apr.. 2020.

²³ "npm | build amazing things." <https://www.npmjs.com/>. Opened 30 apr.. 2020.

6.3 Experiment Analysis

RQ2 conclusion

CPU Usage (Max peak)



6.3.1 CPU Usage

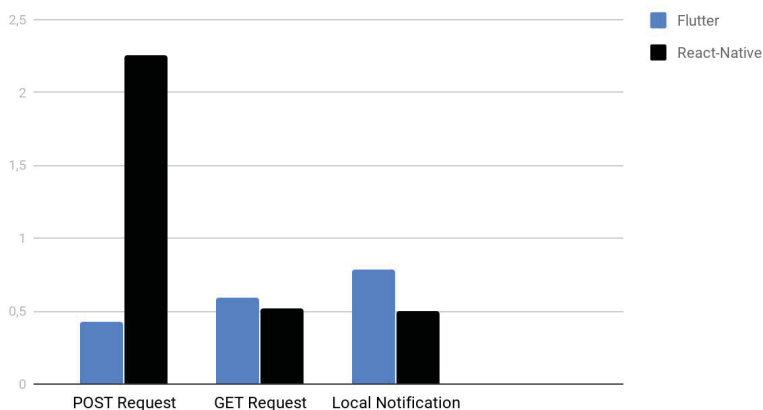
Given the results from chapter five, this diagram shows that flutter clearly has an advantage in CPU usage compared to React-Native.

During the tests performed Flutter had an average of 2% less CPU used during the test cases.

The biggest difference is located

in the stopwatch where Flutter only used CPU every second the timer is counted, unlike React-Native which always had active CPU.

Execution Times (Seconds)



6.3.2 Execution Time

Given the results from chapter five, the difference between flutter and react-native in duration of execution time.

The experiment results shows the difference between Flutter and React Native is in milliseconds and React Native execute the

functionality slower than Flutter despite the CPU advantage in results in 6.3.1. In the POST request there is a big gap between them but there must be a implementation problem because it just too big to be realistic. More testing would be necessary to investigate what problem there can be in the POST request if its a actual accurate result or its corrupted because of an implementation bug.

6.2 Cross-platform vs Platform-specific

RQ3 conclusion

The literature review gave us the result that choosing a cross-platform approach or native approach depends on what kind of project is being developed. There are a few pros and cons of both techniques, but coming up with a general "winner" is difficult, because it varies too much depending on the concept of the app.

6.2.1 Native pros:

A native approach will be more responsive compared to a cross-platform application. When developing advanced apps with complex functionality, the native approach might be better.

6.2.2 Native cons:

Primary cost, if an application is aimed to target both iOS and Android, the entire application basically needs to be implemented twice.

6.2.3 Cross-platform pros:

A cross-platform approach will be faster to get to the market because the framework will target both iOS and Android. The design of a cross-platform app will be more consistent across all the platforms because of this.

6.2.4 Cross-platform cons:

Difficulties with shifting between versions because it needs to focus on both iOS and Android when updating to newer versions. Performance compared to native apps.

Then there are also a few areas which we found out to be very similar to both of the approaches. Testing, for example, is or the most part one streamlined process regardless of different target platforms. And implementing design from a blueprint will be very similar for both native and cross-platform.

Overall we believe that cross-platform is better to use when learning to develop applications and doing smaller apps with shorter life periods. And if the development requires more complex functionality, then the native approach is the best.

7. Conclusion

The mobile application market is growing every year and is estimated to grow even more in the next following years. Because of this, companies need to create even smarter and faster apps, but the knowledge of knowing which framework to use during the development of a mobile app can be hard. Currently, there are two main platforms (Android and iOS), and to make sure a new app is targeting both the platforms there are two general approaches, the native approach or the cross-platform approach.

Both these approaches have their pros and cons which we discuss in our thesis, but we also want to focus on two very popular cross-platform frameworks to come to the conclusion which is the best.

Both Flutter and React Native have a similar approach when it comes to functionality of the application, they both use external packages which are easily installed and implemented. Debugging and testing are fairly similar as well, there are some framework-specific debugging tools that can be used, but other than that the process is the same. Code complexity is mostly decided on external packages. Some packages are simple and some require more configuration. Both Flutter and React Native have built-in tools to help with version control. Flutter uses console commands which can be configured to update specific versions or automatically update the entire project. React Native has a similar function, but also an approach to manually upgrade the packages, this could be very helpful if there is a package which has high coupling in the project.

The tests we did result in a slight advantage to Flutter in overall performance. But React Native was not far behind. But in a more complex application, the results would differentiate even more, if you would follow our results.

Both approaches have their own advantages and it depends a lot on the concept of the app. A native approach is more responsive compared to a cross-platform app, so when developing more complex apps this approach is superior. However a cross-platform app will be faster to get to the market because of the framework target both iOS and Android.

Overall we believe that cross-platform is better to use when learning to develop applications and doing smaller apps with shorter life periods. And if the development requires more complex functionality, then the native approach is the best.

8. Validity Threats

Slight incorrect performance results

While we performed the performance tests for the two frameworks we used Android Profiler for Android and because we had no prior experience with these tools the results could maybe have been more precise if correctly configured.

Another flaw is that we performed the tests 5 times per test, if we wanted more exact results the tests would be performed many more times, and under certain circumstances, such as, under normal operations, under a stress test, etc. This would have resulted in a more exact test result.

The external packages we imported in our projects were not tested with similar packages. Some packages might have a better performance than the ones we used.

Possible database interference

The database we set up was done by our selves in our network. All the fetches and posts to the database were done over an open port in our network, this could impact the latency of the database responses and affect our performance results.

9. Future Work

Functionality variety

We have exclusively covered the performance on the functionality we set out to create when developing the app. But there are many more performance tests that can be done regarding other functionality. A few interesting functionalities that could be tested would be, sensors, encryption, camera functions (artificial reality or video), VoIP(voice over internet protocol) calls. To do a very detailed performance test between the frameworks then all these functions would be tested and under different conditions also.

iOS

We developed our applications to target the Android OS mainly because we did not have enough time and lacked the resources required such as an Apple computer with Xcode environment to develop the iOS version. For those interested in knowing the performance in iOS would have to port the applications we did to an iOS device and run performance tests there.

Device variety

We did the tests on our telephones because that was the only device we could use during this time. But a future work could be to run the tests on different devices to see if the performance would differ.

10. References

1. Wikipedia, "React Native", https://en.wikipedia.org/wiki/React_Native, 2020-04-28
2. Wikipedia, "Flutter (software)", [https://en.wikipedia.org/wiki/Flutter_\(software\)](https://en.wikipedia.org/wiki/Flutter_(software)), 2020-04-29
3. Purrweb, "14 Best React Native apps in 2020", <https://www.purrweb.com/blog/14-best-react-native-apps-in-2020/>, 2020-04-29
4. TechAhead, "Top innovative mobile apps built with flutter", <https://www.techaheadcorp.com/blog/apps-built-with-flutter/>, 2020-04-29
5. Statista, "Mobile app revenues", <https://www.statista.com/statistics/269025/worldwide-mobile-app-revenue-forecast/>, 2020-04-29
6. BuildFire, "Mobile App Download and Usage Statistics", <https://buildfire.com/app-statistics/>, 2020-05-02
7. Diva-portal, "Comparison between Native and Cross-Platform Apps", <http://www.diva-portal.se/smash/get/diva2:795781/FULLTEXT01.pdf>, 2020-04-30
8. Diva-portal, "Cross-Platform Mobile Development", <http://www.diva-portal.org/smash/get/diva2:664680/FULLTEXT01.pdf>, 2020-05-07
9. Diva-portal, "A comparison between native Android and React Native", <http://www.diva-portal.org/smash/get/diva2:998793/FULLTEXT02.pdf>, 2020-05-07
10. Diva-portal, "Riktlinjer för mobil applikationsutveckling", <http://www.diva-portal.org/smash/get/diva2:831096/FULLTEXT01.pdf>, 2020-05-07
11. OATD, "Native-like Cross-Platform Mobile Development", https://www.theseus.fi/bitstream/handle/10024/148975/thesis_Jakhongir_Fayzullaev.pdf?sequence=1&isAllowed=y, 2020-05-09
12. Diva-portal, "Mobilapplikationsutveckling med en användarcentrerad designfilosofi", <http://www.diva-portal.org/smash/get/diva2:1308561/FULLTEXT01.pdf>, 2020-05-13
13. Lund University Publications, "Studie av Cross Platform teknik för mobil applikationsutveckling" <http://lup.lub.lu.se/search/http://lup.lub.lu.se/luur/download?func=downloadFile&recordId=8994411&fileId=8995330>, 2020-05-13
14. Lund University Publications, "Evaluation Targeting React Native in Comparison to Native Mobile Development", <http://lup.lub.lu.se/luur/download?func=downloadFile&recordId=8886469&fileId=8886473>, 2020-05-13
15. Chalmers, "Cross-platform development of mobile applications", <http://publications.lib.chalmers.se/records/fulltext/167502.pdf>, 2020-05-14
16. Medium, "Choosing the best cross-platform framework", <https://medium.com/@tomasz.babiuk/choosing-the-best-cross-platform-framework-for-sharing-code-between-ios-and-android-fa45b8162c81>, 2020-05-14

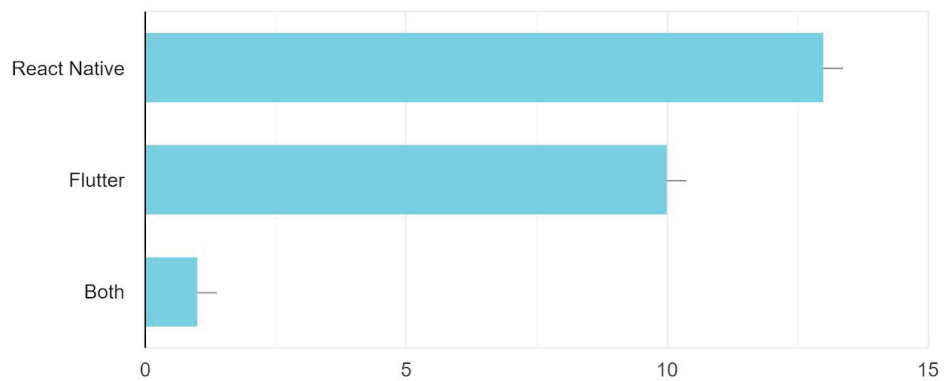
17. Diva-portal, “Recreating a Native Application in React Native”,
<http://www.diva-portal.org/smash/get/diva2:1213248/FULLTEXT01.pdf>, 2020-05-14

11. Annexes

Survey results:

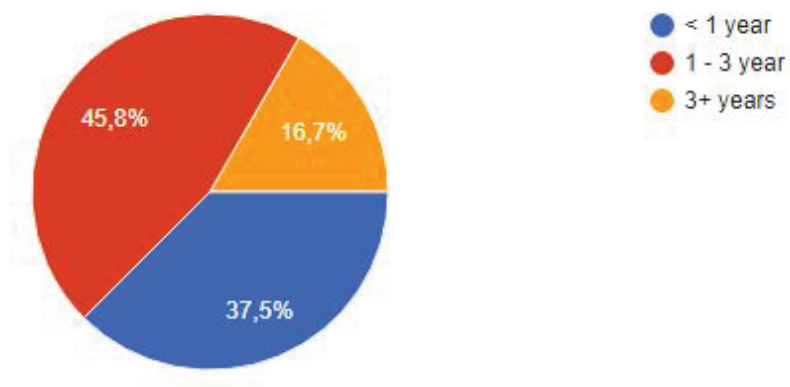
What technology are you most familiar with?

24 svar



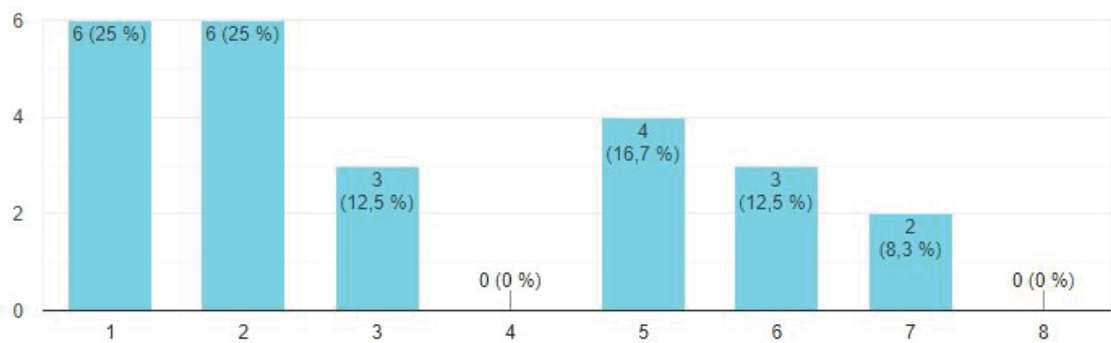
How long have you been working with your chosen technology?

24 svar



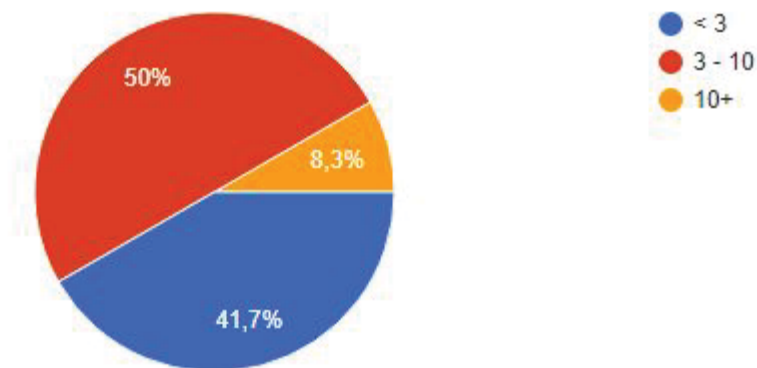
How would you rate the documentation with the chosen technology?

24 svar



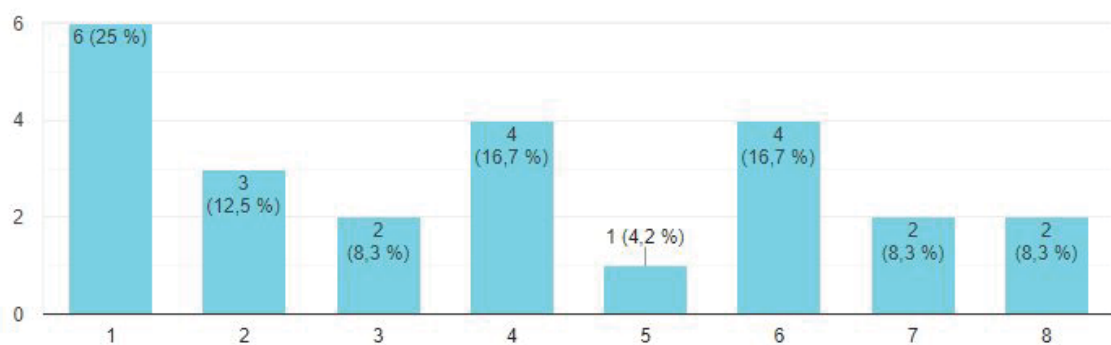
How many projects have you done with the chosen technology?

24 svar



How easy is it to debug a project with the chosen technology with built-in tools?

24 svar



- How do you debug a project? Any external debugging tools?

console.log

React Native Debugger, Flipper

Print lines in console

Mostly Chrome, sometimes Electron or Redux/mobx Dev tools

Emulator plus console error messages tend to suffice, but the debugger, if needed, is more than adequate.

N/A

React Native Debugger

Reactotron, or Flipper

with the stand-alone react-native-debugger

If it's a UI issue then i'll use Android Studio's Flutter Inspector. Otherwise simply setting breakpoints and stepping through code.

Print statements;)

Stackoverflow

Just logs

usually just console.warn, but sometimes i use the built in debugger (that shows on browser inspector) or reactotron

VS Code

Console commands

- Both React Native and Flutter uses external packages to import functionalities. Do you think this is a good method? Why?

Yes, healthy ecosystem, slim core.

Yes, Flutter have made many usefull packages.

No I think it's good. External packages mean a lot potential since the community is working hard. But use it wisely.

Yes, allows for community packages.

Generally yes, because of the extended number of packages and community maintenance for popular packages.

Not in the long run. You'll eventually have issues with libraries not being maintained and would need to look for alternatives.

This is a good method because these technologies do not stuck on base development, and sometimes community driven developments are better

For flutter yes, for react native god fucking no I've heard so many horror stories

That reduces the uniqueness of them as a language

Yes, you get only the libs you need in your project

yes, that way we can leave updates and maintenance to the authors of these packages

Yes, strong community Support.

Yes, keep the core lightweight.

Yes, can find many community made good packages which can help you in your project

Yes

Yes.

I feel it's better to leave it up to the developer to pick the best tool for each use case whether it be an external package or an internal module

This way the framework only delivers the bare minimum and gives you freedom of choice

- Is there something that you feel is missing in your technology compared to other similar technologies that you know of?

Reliable breakpoint debugging.

Designing is hard in Flutter, i can be a big clutter when designing something big.

Of course. React-native, or Flutter, they're still cross-platform, so in some ways, it cannot be like native apps

Easier responsive layout guides.

Most of my main issues I've encountered with RN have been addressed or

Android performance is lacking compared to iOS

JavaScript engine is heavy and slow

Proper design patterns, bloc is great but idk

React naive is very easy to understand. But debugging is very difficult as there is not much support out there and very less know about the errors arising while coding

It's not Native

easier way to make animations and to debug (maybe flipper will fix that on v0.62)

No

Yes, the ability to see interactions between the js runtime and the native environment.

Lack of base packages, needs to implement a lot of packages through NPM in order to make something solid.

Nt

With the freedom of choice the technology gives you, it can be easily overwhelming to get started using it and not staying frozen by the abundance of choice

Finding a straightforward way to do something when you're only getting into RN can be pretty hard at first

- Would you recommend others to learn your chosen technology? Why? Why not?

Yes

Yes, I feel that Flutter has a good potential in the future.

Yes, i think both are really good platform to dig in.

For applicable use cases, RN can build performant cross platform apps. Tools like Expo are progressing toward shared web/mobile code and React developers transition to mobile apps without learning multiple native languages.

Yes, it's fairly easy to pick up the framework and setting up your environment takes little to no time.

I do, because it relies on JavaScript, which is a standard for the web nowadays

But Dart would be the future

Absolutely, far more robust and stable compared to React Native. Plus much more support and active development from Google.

Yes fuck yea, no JavaScript is sold already plus no having to deal with janky third party libraries and dependency hell. Also it's much faster

Yes, it's easy to understand, but the community is very small. I hope it gets as famous as Android Studio so that a lot of solutions will come and doubts will be clarified.

No, of course not. Go learn Android and iOS native

yes, you definitely can't make apps as good as native ones, but for simpler projects on a lower budget it is a good choice. Very easy to learn and has an active community

Yes. Easy, powerful, great developer experience

Yes, large active community.

Yes, its a good framework when learning doing your first application.

No

Link to the GitHub application experiments

<https://github.com/BTH-Bachelor-2020-Simon-och-Hampus>